1. Python Program to Solve 8-Puzzle Problem

Aim

To develop a Python program that solves the **8-Puzzle Problem using Breadth-First Search (BFS)** to find the shortest sequence of moves from the initial state to the goal state.

Algorithm

1. Represent the 8-puzzle as a 3×3 matrix.
2. Define the initial state and goal state.
3. Represent the empty tile using `0`.
4. Locate the position of the empty tile.
5. Generate all possible moves of the empty tile: up, down, left and right.
6. Use Breadth-First Search (BFS) to explore the possible states.
7. Maintain a visited set to avoid repeated states.
8. Store the parent of each state to reconstruct the solution path.
9. If the goal state is reached, display the sequence of moves.
10. If all possible states are explored without reaching the goal, report that no solution exists.

Program

```
from collections import deque

def print_puzzle(state):
    for i in range(0, 9, 3):
        print(state[i:i + 3])
    print()


def get_neighbors(state):
    neighbors = []

    zero = state.index(0)
    row = zero // 3
    col = zero % 3

    moves = [
```

```
        (-1, 0),   # Up
        (1, 0),    # Down
        (0, -1),   # Left
        (0, 1)     # Right
    ]

    for dr, dc in moves:
        new_row = row + dr
        new_col = col + dc

        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_zero = new_row * 3 + new_col

            new_state = list(state)
            new_state[zero], new_state[new_zero] = \
                new_state[new_zero], new_state[zero]

            neighbors.append(tuple(new_state))

    return neighbors


def solve_8_puzzle(initial, goal):
    queue = deque([initial])
    visited = {initial}
    parent = {initial: None}

    while queue:
        current = queue.popleft()

        if current == goal:
            path = []

            while current is not None:
                path.append(current)
                current = parent[current]

            path.reverse()
            return path

        for neighbor in get_neighbors(current):
            if neighbor not in visited:
                visited.add(neighbor)
                parent[neighbor] = current
```

```
            queue.append(neighbor)

    return None


# Initial state
initial = (
    1, 2, 3,
    4, 0, 6,
    7, 5, 8
)

# Goal state
goal = (
    1, 2, 3,
    4, 5, 6,
    7, 8, 0
)

solution = solve_8_puzzle(initial, goal)

if solution:
    print("Solution found!")
    print("Number of moves:", len(solution) - 1)

    for step, state in enumerate(solution):
        print("Step", step)
        print_puzzle(state)
else:
    print("No solution exists.")
```

Result

Thus, the **8-Puzzle Problem was solved successfully using Python and Breadth-First Search (BFS).**

IDLE Shell 3.13.2

File Edit Shell Debug Options Window Help

```
Python 3.13.2 (tags/v3.13.2:4f8bb39, Feb  4 2025, 15:23:48) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Prerna Premkumar/OneDrive/Desktop/ARTIFICIAL INTELLIGENCE/Exp-1 8-Puzzle.py
Solution found!
Number of moves: 2
Step 0
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

Step 1
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

Step 2
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)

>>>
```

2. Python Program to Solve 8-Queen Problem

Aim

To develop a Python program that solves the **8-Queen Problem using the Backtracking algorithm**, such that no two queens attack each other.

Algorithm

1. Represent the chessboard as an 8×8 matrix.
2. Place one queen in each row.
3. Start placing queens from the first row.
4. Check whether the current position is safe.
5. A position is safe if no queen exists in the same column.
6. Check both diagonals to ensure that no queen attacks the current position.
7. If the position is safe, place a queen.
8. Move to the next row and repeat the process.
9. If no safe position is available, backtrack to the previous row.
10. Continue until queens are placed in all 8 rows.
11. Display the solution.

Program

```
N = 8

board = [[0 for _ in range(N)] for _ in range(N)]


def is_safe(row, col):
    # Check column
    for i in range(row):
        if board[i][col] == 1:
            return False

    # Check upper-left diagonal
    i = row - 1
    j = col - 1

    while i >= 0 and j >= 0:
        if board[i][j] == 1:
```

```
            return False
        i -= 1
        j -= 1

    # Check upper-right diagonal
    i = row - 1
    j = col + 1

    while i >= 0 and j < N:
        if board[i][j] == 1:
            return False
        i -= 1
        j += 1

    return True


def solve_queens(row):
    # All queens are placed
    if row == N:
        return True

    # Try every column
    for col in range(N):

        if is_safe(row, col):
            board[row][col] = 1

            if solve_queens(row + 1):
                return True

            # Backtrack
            board[row][col] = 0

    return False

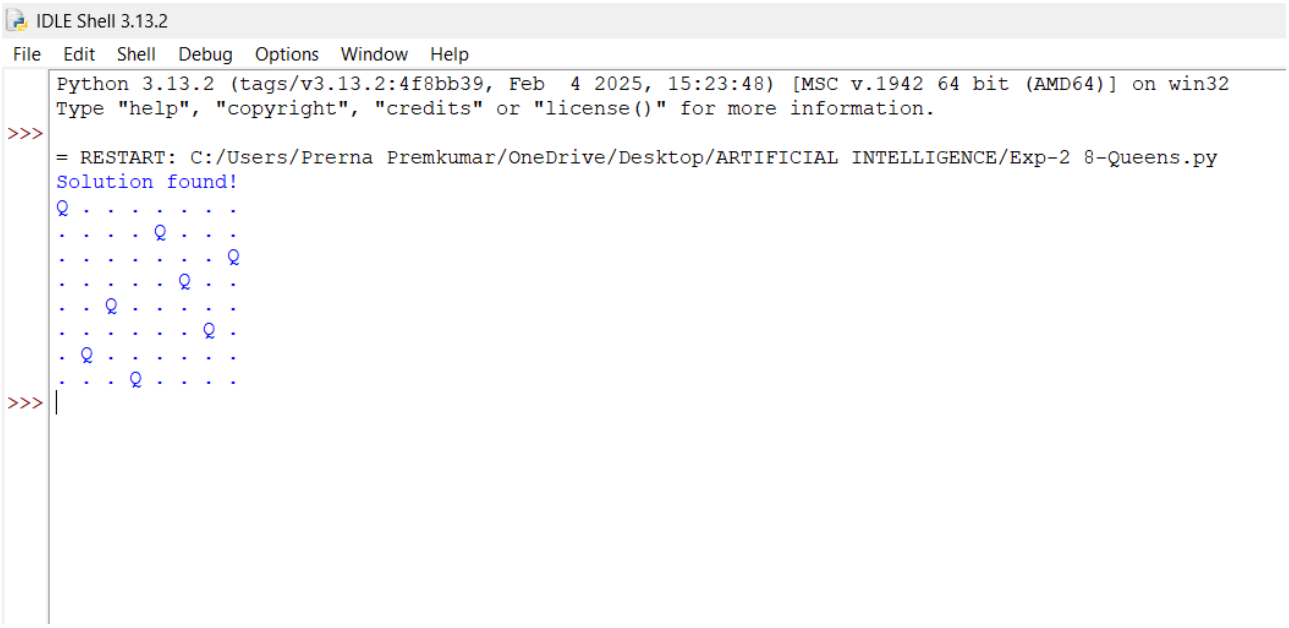

def print_board():
    for row in board:
        for cell in row:
            if cell == 1:
                print("Q", end=" ")
            else:
                print(".", end=" ")
```

```
        print()


if solve_queens(0):
    print("Solution found!")
    print_board()
else:
    print("No solution exists.")
```

Result

Thus, the **8-Queen Problem was solved successfully using Python and the Backtracking algorithm.**



3. Python Program for Water Jug Problem

Aim

To develop a Python program to solve the **Water Jug Problem using Breadth-First Search (BFS)** by finding a sequence of operations to obtain the required amount of water.

Algorithm

1. Define the capacities of the two water jugs.
2. Define the target amount of water.
3. Represent the state as `(jug1, jug2)`, where `jug1` and `jug2` represent the current amounts of water.
4. Start with both jugs empty `(0, 0)`.
5. Generate all possible operations:
    - Fill Jug 1.
    - Fill Jug 2.
    - Empty Jug 1.
    - Empty Jug 2.
    - Pour water from Jug 1 to Jug 2.
    - Pour water from Jug 2 to Jug 1.
6. Use BFS to explore all possible states.
7. Maintain a visited set to avoid visiting the same state repeatedly.
8. Store the parent state and operation for each new state.
9. If either jug contains the target amount, reconstruct the solution path.
10. If all possible states are explored without reaching the target, report that no solution exists.

Program

```
from collections import deque


def water_jug(capacity1, capacity2, target):
    initial_state = (0, 0)

    queue = deque([initial_state])
    visited = {initial_state}

    parent = {
        initial_state: None
    }
```

```
action = {
   initial_state: "Start"
}

while queue:
   jug1, jug2 = queue.popleft()

   # Check whether target is reached
   if jug1 == target or jug2 == target:

      path = []
      state = (jug1, jug2)

      while state is not None:
         path.append((state, action[state]))
         state = parent[state]

      path.reverse()
      return path

   states = []

   # Fill Jug 1
   states.append(
      ((capacity1, jug2), "Fill Jug 1")
   )

   # Fill Jug 2
   states.append(
      ((jug1, capacity2), "Fill Jug 2")
   )

   # Empty Jug 1
   states.append(
      ((0, jug2), "Empty Jug 1")
   )

   # Empty Jug 2
   states.append(
      ((jug1, 0), "Empty Jug 2")
   )

   # Pour Jug 1 into Jug 2
```

```
        amount = min(jug1, capacity2 - jug2)

        states.append(
            (
                (jug1 - amount, jug2 + amount),
                "Pour Jug 1 -> Jug 2"
            )
        )

        # Pour Jug 2 into Jug 1
        amount = min(jug2, capacity1 - jug1)

        states.append(
            (
                (jug1 + amount, jug2 - amount),
                "Pour Jug 2 -> Jug 1"
            )
        )

        # Add new states to queue
        for new_state, operation in states:

            if new_state not in visited:
                visited.add(new_state)
                parent[new_state] = (jug1, jug2)
                action[new_state] = operation
                queue.append(new_state)

    return None


# Jug capacities
capacity1 = 4
capacity2 = 3

# Required amount
target = 2

solution = water_jug(capacity1, capacity2, target)

if solution:
    print("Solution found!")

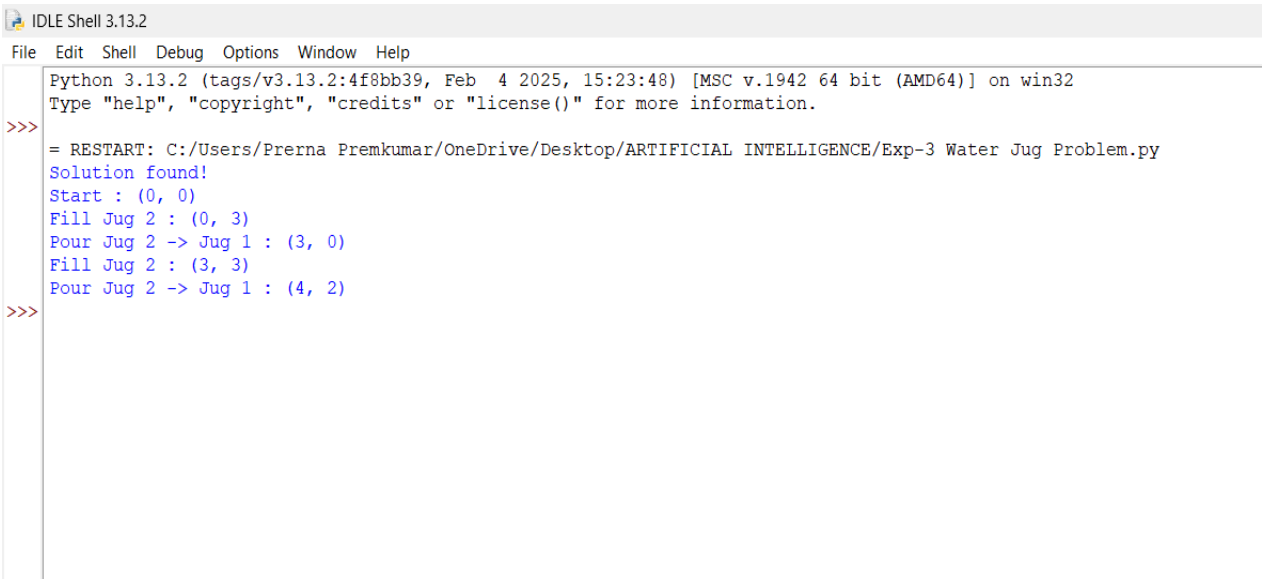
    for state, operation in solution:
```

```
        print(operation, ":", state)

else:
    print("No solution exists.")
```

Result

Thus, the **Water Jug Problem was solved successfully using Python and Breadth-First Search (BFS).**



4. Python Program for Crypt-Arithmetic Problem

Aim

To develop a Python program to solve a **Crypt-Arithmetic Problem** by assigning unique digits to the letters such that the given arithmetic equation is satisfied.

Algorithm

1. Take a crypt-arithmetic equation such as `SEND + MORE = MONEY`.
2. Identify all unique letters present in the equation.
3. Assign a different digit from 0 to 9 to each letter.
4. Ensure that the first letter of each word is not assigned zero.
5. Substitute the assigned digits into the words.
6. Convert the words into their corresponding numerical values.
7. Check whether the arithmetic equation is satisfied.
8. If the equation is satisfied, display the solution.
9. Otherwise, try another possible digit assignment.
10. Continue until a valid solution is found.

Program

```
from itertools import permutations


def solve_cryptarithmetic():
    letters = "SENDMORY"

    for digits in permutations(range(10), len(letters)):
        S, E, N, D, M, O, R, Y = digits

        # First letters cannot be zero
        if S == 0 or M == 0:
            continue

        SEND = 1000 * S + 100 * E + 10 * N + D
        MORE = 1000 * M + 100 * O + 10 * R + E
        MONEY = (
            10000 * M +
            1000 * O +
```

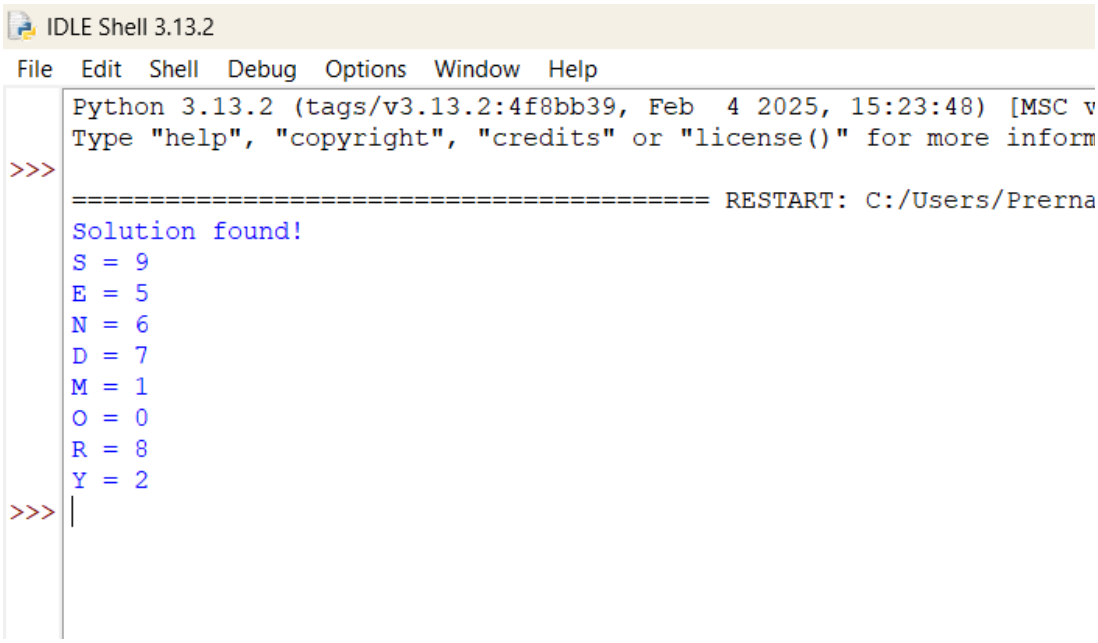
```
        100 * N +
        10 * E +
        Y
    )

    if SEND + MORE == MONEY:
        print("Solution found!")
        print("S =", S)
        print("E =", E)
        print("N =", N)
        print("D =", D)
        print("M =", M)
        print("O =", O)
        print("R =", R)
        print("Y =", Y)
        return


solve_cryptarithmetic()
```

Result

Thus, the **Crypt-Arithmetic Problem was solved successfully using Python.**



5. Python Program for Missionaries and Cannibals Problem

Aim

To develop a Python program to solve the **Missionaries and Cannibals Problem using Breadth-First Search (BFS)** while ensuring that the missionaries are never outnumbered by cannibals on either side of the river.

Algorithm

1. Represent a state as `(M_left, C_left, boat)`.
2. Initially, there are 3 missionaries and 3 cannibals on the left side, with the boat on the left.
3. The goal is to move all missionaries and cannibals to the right side.
4. The boat can carry one or two people at a time.
5. Generate all possible boat movements.
6. Check whether each new state is valid.
7. A state is valid if missionaries are not outnumbered by cannibals on either side, whenever missionaries are present.
8. Use BFS to explore all possible states.
9. Maintain a visited set to avoid repeated states.
10. Store parent states to reconstruct the solution path.
11. Stop when the goal state is reached.
12. Display the sequence of valid states.

Program

```
from collections import deque


def is_valid(state):
    missionaries_left, cannibals_left, boat = state

    missionaries_right = 3 - missionaries_left
    cannibals_right = 3 - cannibals_left

    # Check valid range
    if not (0 <= missionaries_left <= 3):
        return False
```

```
    if not (0 <= cannibals_left <= 3):
        return False

    # Check left side
    if missionaries_left > 0:
        if missionaries_left < cannibals_left:
            return False

    # Check right side
    if missionaries_right > 0:
        if missionaries_right < cannibals_right:
            return False

    return True


def solve_missionaries_cannibals():

    initial = (3, 3, 0)
    goal = (0, 0, 1)

    queue = deque([initial])
    visited = {initial}

    parent = {initial: None}

    # Possible combinations of people in the boat
    moves = [
        (1, 0),  # One missionary
        (2, 0),  # Two missionaries
        (0, 1),  # One cannibal
        (0, 2),  # Two cannibals
        (1, 1)   # One missionary and one cannibal
    ]

    while queue:

        state = queue.popleft()

        if state == goal:
            path = []

            while state is not None:
                path.append(state)
```

```
            state = parent[state]

        path.reverse()
        return path

    missionaries, cannibals, boat = state

    for m, c in moves:

        if boat == 0:
            # Boat moves from left to right
            new_state = (
                missionaries - m,
                cannibals - c,
                1
            )

        else:
            # Boat moves from right to left
            new_state = (
                missionaries + m,
                cannibals + c,
                0
            )

        if is_valid(new_state) and new_state not in visited:

            visited.add(new_state)
            parent[new_state] = state
            queue.append(new_state)

  return None


solution = solve_missionaries_cannibals()

if solution:
  print("Solution found!")

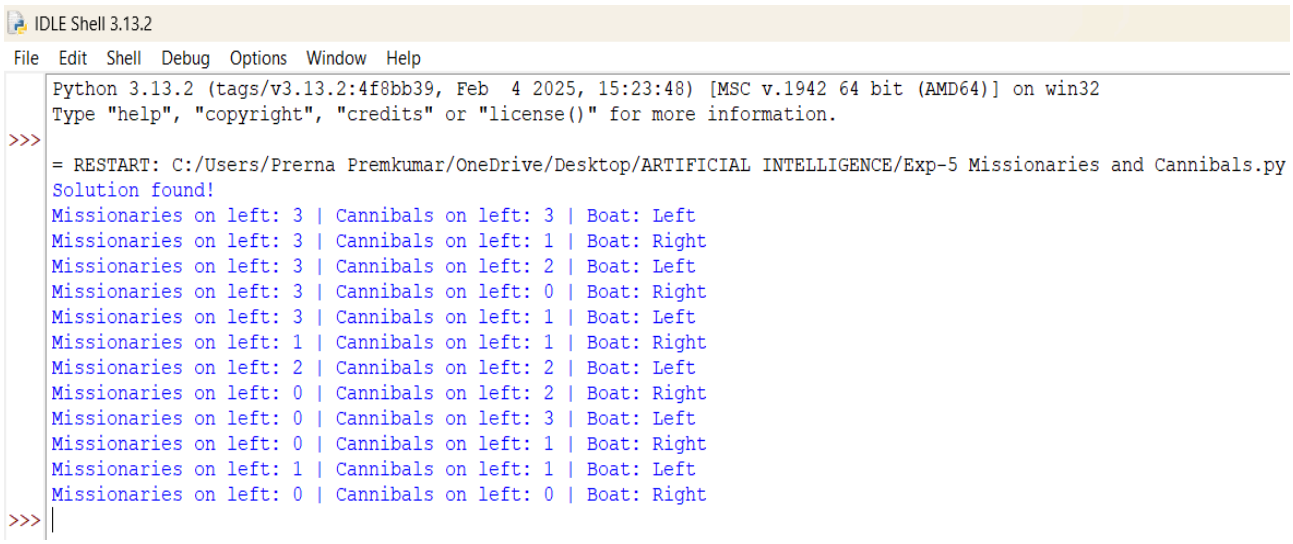
  for state in solution:
    missionaries, cannibals, boat = state

    side = "Left" if boat == 0 else "Right"
```

```
        print(
            "Missionaries on left:",
            missionaries,
            "| Cannibals on left:",
            cannibals,
            "| Boat:",
            side
        )
else:
    print("No solution exists.")
```

Result

Thus, the **Missionaries and Cannibals Problem was solved successfully using Python and Breadth-First Search (BFS).**



6. Python Program for Vacuum Cleaner Problem

Aim

To develop a Python program to solve the **Vacuum Cleaner Problem** using a state-space search approach by moving the vacuum cleaner between rooms and cleaning the dirty rooms.

Algorithm

1. Represent the environment using two rooms: Room A and Room B.
2. Each room can be either clean or dirty.
3. Represent the vacuum cleaner's position as either Room A or Room B.
4. Define the initial state of the rooms and the vacuum cleaner.
5. If the current room is dirty, perform the **Suck** operation.
6. If the current room is clean, move the vacuum cleaner to the other room.
7. Continue the process until both rooms are clean.
8. Keep track of visited states to avoid repeated operations.
9. Display each state and operation performed.
10. Stop when the goal state, where both rooms are clean, is reached.

Program

```
from collections import deque


def solve_vacuum_cleaner(initial_state):

    # State format:
    # (Room A, Room B, Vacuum Position)
    # 1 = Dirty
    # 0 = Clean

    goal = (0, 0, initial_state[2])

    queue = deque([initial_state])
    visited = {initial_state}

    parent = {initial_state: None}
    action = {initial_state: "Start"}
```

```
while queue:

    state = queue.popleft()

    room_a, room_b, position = state

    # Check goal
    if room_a == 0 and room_b == 0:

        path = []

        while state is not None:
            path.append((state, action[state]))
            state = parent[state]

        path.reverse()
        return path

    possible_moves = []

    # Vacuum is in Room A
    if position == "A":

        if room_a == 1:
            # Clean Room A
            possible_moves.append(
                ((0, room_b, "A"), "Suck")
            )

        else:
            # Move to Room B
            possible_moves.append(
                ((room_a, room_b, "B"), "Move Right")
            )

    # Vacuum is in Room B
    else:

        if room_b == 1:
            # Clean Room B
            possible_moves.append(
                ((room_a, 0, "B"), "Suck")
            )
```

```
            else:
                # Move to Room A
                possible_moves.append(
                    ((room_a, room_b, "A"), "Move Left")
                )

        for new_state, operation in possible_moves:

            if new_state not in visited:
                visited.add(new_state)

                parent[new_state] = state
                action[new_state] = operation

                queue.append(new_state)

    return None


# Initial state
# 1 = Dirty
# 0 = Clean
initial_state = (1, 1, "A")

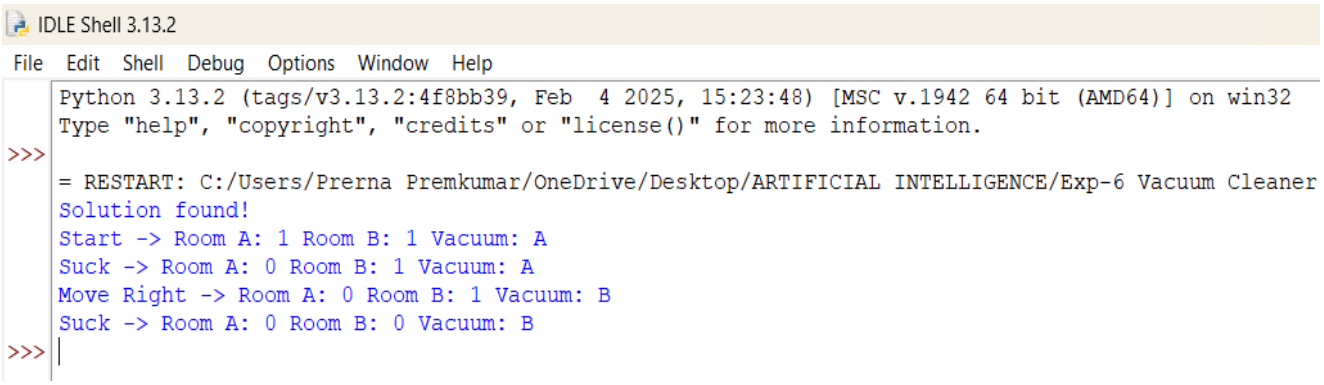
solution = solve_vacuum_cleaner(initial_state)

if solution:
    print("Solution found!")

    for state, operation in solution:
        print(
            operation,
            "->",
            "Room A:", state[0],
            "Room B:", state[1],
            "Vacuum:", state[2]
        )
else:
    print("No solution exists.")
```

Result

Thus, the **Vacuum Cleaner Problem was solved successfully using Python and a state-space search approach.**


IDLE Shell 3.13.2
File Edit Shell Debug Options Window Help
Python 3.13.2 (tags/v3.13.2:4f8bb39, Feb 4 2025, 15:23:48) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Prerna Premkumar/OneDrive/Desktop/ARTIFICIAL INTELLIGENCE/Exp-6 Vacuum Cleaner
Solution found!
Start -> Room A: 1 Room B: 1 Vacuum: A
Suck -> Room A: 0 Room B: 1 Vacuum: A
Move Right -> Room A: 0 Room B: 1 Vacuum: B
Suck -> Room A: 0 Room B: 0 Vacuum: B
>>>

7. Python Program to Implement BFS

Aim

To develop a Python program to implement **Breadth-First Search (BFS)** for traversing a graph by visiting vertices level by level.

Algorithm

1. Represent the graph using an adjacency list.
2. Select the starting vertex.
3. Create an empty queue and a visited set.
4. Add the starting vertex to the queue and mark it as visited.
5. Remove the first vertex from the queue.
6. Visit and display the vertex.
7. Add all its unvisited adjacent vertices to the queue.
8. Mark the newly added vertices as visited.
9. Repeat the process until the queue becomes empty.
10. The vertices visited in this order form the BFS traversal.

Program

```
from collections import deque


def bfs(graph, start):
    visited = set()
    queue = deque()

    visited.add(start)
    queue.append(start)

    print("BFS Traversal:")

    while queue:
        vertex = queue.popleft()

        print(vertex, end=" ")

        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
```

```
            queue.append(neighbor)

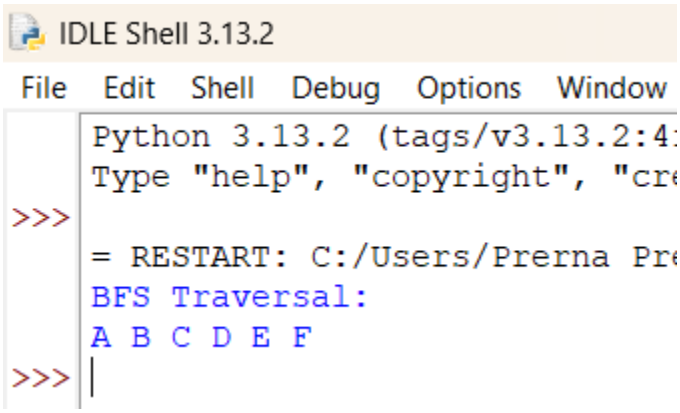

# Graph representation
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}

# Starting vertex
start = 'A'

bfs(graph, start)
```

Result

Thus, **Breadth-First Search (BFS) was implemented successfully using Python.**



8. Python Program to Implement DFS

Aim

To develop a Python program to implement **Depth-First Search (DFS)** for traversing a graph by exploring as far as possible along each branch before backtracking.

Algorithm

1. Represent the graph using an adjacency list.
2. Select the starting vertex.
3. Create an empty set to store visited vertices.
4. Mark the current vertex as visited.
5. Display the current vertex.
6. Visit each unvisited adjacent vertex recursively.
7. Continue until there are no more unvisited adjacent vertices.
8. Backtrack to the previous vertex.
9. Repeat the process until all reachable vertices are visited.
10. The vertices visited in this order form the DFS traversal.

Program

```
def dfs(graph, vertex, visited):

    visited.add(vertex)

    print(vertex, end=" ")

    for neighbor in graph[vertex]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)


# Graph representation
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}
```

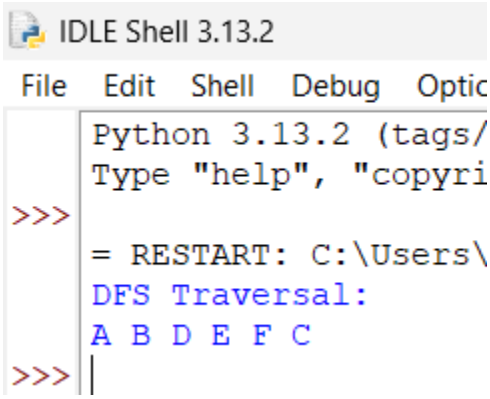
```
# Starting vertex
start = 'A'

visited = set()

print("DFS Traversal:")
dfs(graph, start, visited)
```

Result

Thus, **Depth-First Search (DFS) was implemented successfully using Python.**



9. Python Program to Implement Travelling Salesman Problem

Aim

To develop a Python program to solve the **Travelling Salesman Problem (TSP)** by finding the minimum-cost route that visits every city exactly once and returns to the starting city.

Algorithm

1. Represent the cities and their distances using a distance matrix.
2. Select a starting city.
3. Generate all possible permutations of the remaining cities.
4. For each permutation, construct a complete tour.
5. Calculate the total distance of the tour.
6. Add the distance from the last city back to the starting city.
7. Compare the total distance with the minimum distance found so far.
8. Store the route having the minimum total distance.
9. Repeat for all possible routes.
10. Display the minimum-distance route and its total distance.

Program

```
from itertools import permutations


def travelling_salesman(graph, start):
    cities = list(graph.keys())

    # Remove starting city
    cities.remove(start)

    min_distance = float('inf')
    best_route = None

    # Generate all possible routes
    for permutation in permutations(cities):

        route = [start] + list(permutation) + [start]

        total_distance = 0
```

```
        # Calculate total distance
        for i in range(len(route) - 1):
            total_distance += graph[route[i]][route[i + 1]]

        # Check for minimum distance
        if total_distance < min_distance:
            min_distance = total_distance
            best_route = route

    return best_route, min_distance


# Distance matrix
graph = {
    'A': {
        'A': 0,
        'B': 10,
        'C': 15,
        'D': 20
    },

    'B': {
        'A': 10,
        'B': 0,
        'C': 35,
        'D': 25
    },

    'C': {
        'A': 15,
        'B': 35,
        'C': 0,
        'D': 30
    },

    'D': {
        'A': 20,
        'B': 25,
        'C': 30,
        'D': 0
    }
}
```

```
# Starting city
start = 'A'

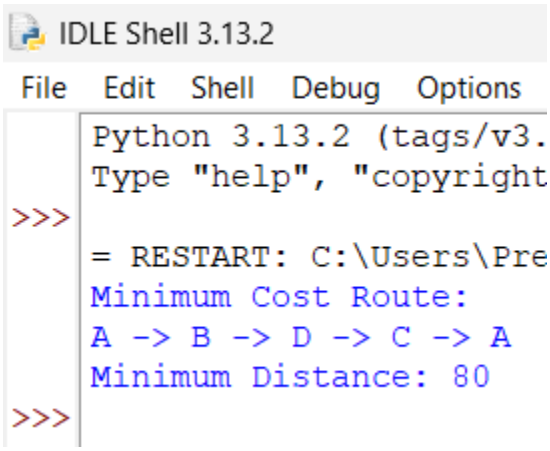
route, distance = travelling_salesman(graph, start)

print("Minimum Cost Route:")
print(" -> ".join(route))

print("Minimum Distance:", distance)
```

Result

Thus, the **Travelling Salesman Problem was solved successfully using Python by finding the minimum-cost tour that visits every city exactly once and returns to the starting city.**



10. Python Program to Implement A* Algorithm

Aim

To develop a Python program to implement the **A* (A-Star) algorithm** for finding the shortest path between a starting node and a goal node using path cost and heuristic cost.

Algorithm

1. Represent the graph using an adjacency list with edge costs.
2. Define a heuristic value for each node.
3. Set the starting node as the current node.
4. Calculate the evaluation function:
   f(n)=g(n)+h(n)f(n) = g(n) + h(n)
   where `g(n)` is the cost from the start node and `h(n)` is the heuristic cost to the goal.
5. Add the starting node to the open list.
6. Select the node having the smallest `f(n)` value.
7. If the selected node is the goal node, reconstruct the path.
8. Otherwise, examine all neighbouring nodes.
9. Calculate the new path cost for each neighbour.
10. Update the cost and parent if a better path is found.
11. Continue until the goal is reached or there are no nodes left to explore.
12. Display the shortest path and its total cost.

Program

```
import heapq


def a_star(graph, heuristic, start, goal):

    # Priority queue
    open_list = []

    heapq.heappush(open_list, (0, start))

    # Cost from start node
    g_cost = {start: 0}
```

```
    # Parent of each node
    parent = {start: None}

    while open_list:

        current_f, current = heapq.heappop(open_list)

        # Goal reached
        if current == goal:

            path = []

            while current is not None:
                path.append(current)
                current = parent[current]

            path.reverse()

            return path, g_cost[goal]

        # Explore neighbours
        for neighbor, cost in graph[current].items():

            new_cost = g_cost[current] + cost

            if neighbor not in g_cost or new_cost < g_cost[neighbor]:

                g_cost[neighbor] = new_cost
                parent[neighbor] = current

                f_cost = new_cost + heuristic[neighbor]

                heapq.heappush(
                    open_list,
                    (f_cost, neighbor)
                )

    return None, None


# Graph
graph = {
    'A': {'B': 1, 'C': 3},
    'B': {'D': 3, 'E': 1},
```

```
    'C': {'F': 2},
    'D': {'G': 2},
    'E': {'G': 3},
    'F': {'G': 1},
    'G': {}
}

# Heuristic values
heuristic = {
    'A': 6,
    'B': 5,
    'C': 4,
    'D': 2,
    'E': 3,
    'F': 1,
    'G': 0
}

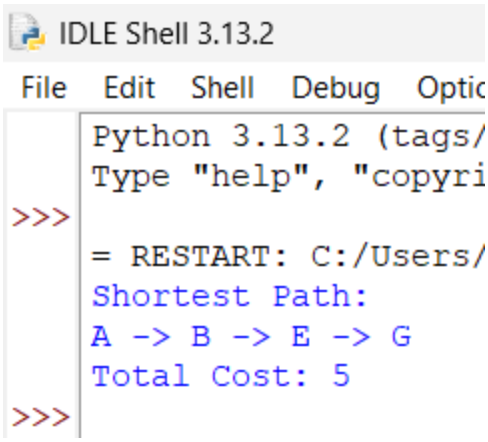
start = 'A'
goal = 'G'

path, cost = a_star(graph, heuristic, start, goal)

if path:
    print("Shortest Path:")
    print(" -> ".join(path))
    print("Total Cost:", cost)
else:
    print("No path exists.")
```

Result

Thus, the **A* algorithm was implemented successfully using Python to find the shortest path between the given starting node and goal node.**


IDLE Shell 3.13.2
File Edit Shell Debug Optic
Python 3.13.2 (tags/
Type "help", "copyri
>>>
= RESTART: C:/Users/
Shortest Path:
A -> B -> E -> G
Total Cost: 5
>>>

11. Python Program for Map Coloring to Implement CSP

Aim

To develop a Python program to solve the **Map Coloring Problem using Constraint Satisfaction Problem (CSP)** by assigning colors to different regions such that no two adjacent regions have the same color.

Algorithm

1. Represent the map as a graph where each region is a variable.
2. Represent adjacent regions using an adjacency list.
3. Define a set of available colors.
4. Assign a color to each region.
5. Check whether the selected color is consistent with the colors assigned to neighbouring regions.
6. If the color is valid, assign it to the region.
7. Move to the next uncolored region.
8. If no valid color is available, backtrack to the previous region.
9. Continue until all regions are assigned a color.
10. Display the valid map coloring.

Program

```
def is_safe(region, color, assignment, graph):

    for neighbor in graph[region]:

        if neighbor in assignment:
            if assignment[neighbor] == color:
                return False

    return True


def map_coloring(regions, colors, graph, assignment):

    # All regions are colored
    if len(assignment) == len(regions):
        return True
```

```
    # Select an uncolored region
    for region in regions:

        if region not in assignment:

            # Try every color
            for color in colors:

                if is_safe(
                    region,
                    color,
                    assignment,
                    graph
                ):

                    assignment[region] = color

                    # Recursively color remaining regions
                    if map_coloring(
                        regions,
                        colors,
                        graph,
                        assignment
                    ):
                        return True

                    # Backtrack
                    del assignment[region]

            return False

    return False


# Map representation
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D'],
    'D': ['B', 'C']
}

regions = list(graph.keys())
```

```
# Available colors
colors = ['Red', 'Green', 'Blue']

assignment = {}

if map_coloring(
    regions,
    colors,
    graph,
    assignment
):

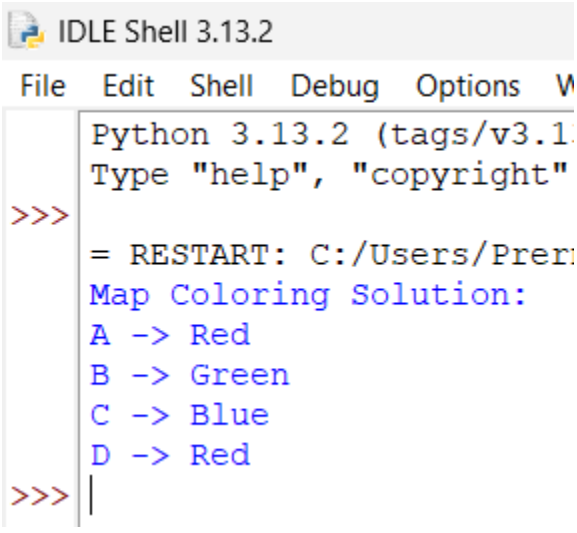
    print("Map Coloring Solution:")

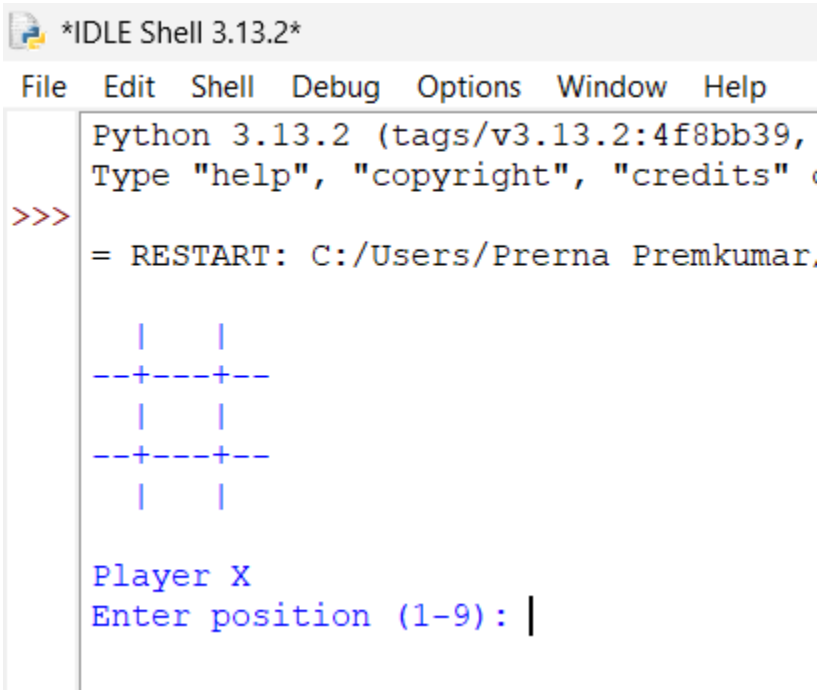
    for region in regions:
        print(region, "->", assignment[region])

else:
    print("No valid coloring exists.")
```

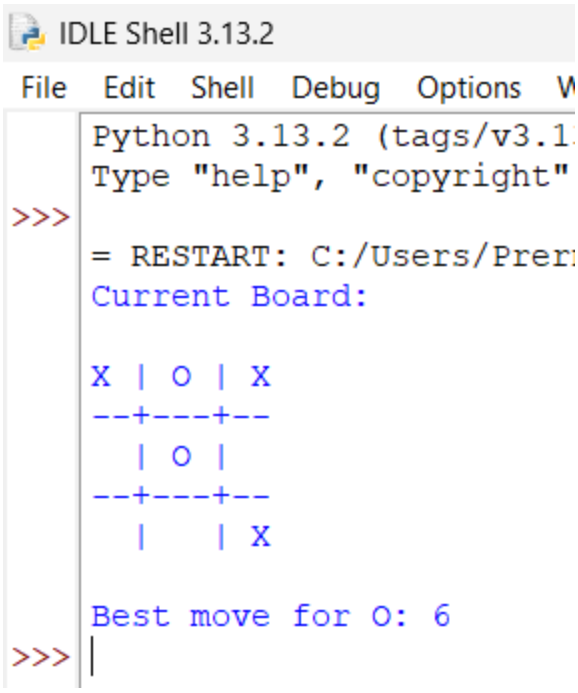
Result

Thus, the **Map Coloring Problem was solved successfully using Python and the Constraint Satisfaction Problem (CSP) approach.**





EXERCISE 13 - MINIMAX ALGORITHM



EXERCISE 14 - ALPHA-BETA PRUNING ALGORITHM

